

第十一章 并发控制 — 选择题解析

第 1 题

答案：B

解析：

初始 $A = 0$ ，逐步追踪：

1. T1: Read(A) → T1 读到 $A=0$

2. T2: Read(A) → T2 读到 $A=0$

3. T1: $A = A + 1$ → T1 算出 $A=1$

4. T1: Write(A) → 写入 $A=1$

5. T2: $A = A + 2$ → T2 用它之前读的 $A=0$ 算出 $A=2$ （注意 T2 没重读）

6. T2: Write(A) → 写入 $A=2$ ，覆盖了 T1 的 $A=1$

最终 $A = 2$ 。T1 的修改 (+1) 被 T2 覆盖丢失了。

B 正确： $A = 2$ ，丢失修改（T1 的修改被覆盖）。选 B。

两个事务都基于“旧的 $A=0$ ”做加法，后写的覆盖先写的——经典的丢失修改问题。根因是修改前没加 X 锁互斥。

第 2 题

答案：C

解析：

- X 锁（排他锁）：加了 X 锁后，其他事务既不能读也不能写（要等锁释放）。
- S 锁（共享锁）：加了 S 锁后，其他事务可以读（也加 S 锁）但不能写。

A 错：X 锁后其他事务不能读，不是“可以读不能写”。

B 错：S 锁后 T 自己只能读不能写（要写得升级成 X 锁）。

C 正确：多个事务可以同时同一数据加 S 锁——这就是“共享”的含义，读读不冲突。选 C。

D 错：X 锁和 S 锁不相容，X 锁独占一切。

S 锁 = 共享书（多人可同时看）；X 锁 = 独占书（只有我能碰，别人看都不行）。

第 3 题

答案：B

解析：

三级封锁协议：

- 一级：修改前加 X 锁，直到事务结束释放 → 防丢失修改
- 二级：一级 + 读前加 S 锁，读完即释放（不持有到事务结束） → 防丢失修改 + 防读脏数据
- 三级：一级 + 读前加 S 锁，直到事务结束释放 → 防丢失修改 + 防读脏数据 + 保证可重复读

A 错：一级只能防丢失修改，不能防读脏数据。

B 正确：二级在一级基础上，读前加 S 锁读完释放，防止读脏数据。选 B。

C 错：三级是读前加 S 锁直到事务结束释放（不是读完释放），才保证可重复读。说“读完释放”是二级不是三级。

D 错：三级确实比二级严格，二级比一级严格，这个递进关系是对的。



区别在 S 锁持有时间：二级读完就放（防脏读），三级持到事务结束（保可重复读）。持得越久越严格。

第 4 题

答案：C

解析：

两段锁协议（2PL）：事务分两个阶段——先加锁阶段（只加不放），再解锁阶段（只放不加）。

A 正确：2PL 分加锁阶段和释放锁阶段。

B 正确：遵守 2PL 的事务调度一定可串行化（这是 2PL 的核心保证）。

C 错误：遵守 2PL 的事务仍可能死锁——2PL 保证可串行化，但不保证无死锁。选 C。

D 正确：释放一个锁后不能再申请新锁（这就是“两段”的分界）。

2PL 的承诺：可串行化。不承诺：无死锁。死锁是 2PL 的代价，得另外用一次封锁法/顺序封锁法预防。

第 5 题

答案：B

解析：

T1 持有 A 的 X 锁，请求 B 的 X 锁；T2 持有 B 的 X 锁，请求 A 的 X 锁。

这是经典的循环等待：T1 等 T2 释放 B，T2 等 T1 释放 A，谁也不让谁。

B 正确：死锁状态。选 B。

A 错：活锁是某个事务一直被优先级高的抢先，永远等下去——这里不是。

C 错：不是正常等待，是循环等待导致永久阻塞。

D 错：可串行化是调度正确性概念，和锁等待状态无关。

死锁 = 循环等待。你锁 A 等 B，我锁 B 等 A，两人互相等永远等不到——经典死锁。

第 6 题

答案：A

解析：

- 活锁：事务一直被其他事务抢先，永远轮不到自己执行。可用先来先服务策略避免。

- 死锁：事务循环等待，需用死锁预防（一次封锁法/顺序封锁法）或诊断（超时法/等待图法）。

A 正确：活锁可通过先来先服务避免——让等待最久的事务优先获得锁。选 A。

B 错：死锁不能靠先来先服务避免，死锁是循环等待，和排队顺序无关。

C 错：活锁不如死锁严重——活锁还有恢复可能（调整调度），死锁是永久卡死需主动解除。

D 错：死锁诊断不只超时法，还有等待图法。

活锁 = 老插队，你永远排不到（先来先服务解决）；死锁 = 互相锁死，谁也动不了（得主动检测解除）。

第 7 题

答案：B

解析：

可串行化分两种：冲突可串行化和视图可串行化。冲突可串行化是可串行化的充分条件，但不是必要条件。

A 错：不是所有可串行化调度都是冲突可串行化——视图可串行化但不冲突可串行化的调度是存在的。



B 正确：冲突可串行化调度一定是可串行化调度（冲突可串行化是更强的条件）。选 B。

C 错：不可串行化调度不一定有死锁，死锁和可串行化是两个独立概念。

D 错：2PL 保证可串行化，不保证无死锁（第 4 题已述）。

包含关系：冲突可串行化 $\supset$ 可串行化。冲突可串行化是可串行化的"加强版"，一定可串行化，但反过来不成立。

第 8 题

答案：B

解析：

T1 第一次读 $A=10$ ，T2 把 A 改成 20 并提交，T1 第二次读 $A=20$ 。

注意：T2 已提交，所以 T1 读到的不是"脏数据"（脏数据是读未提交的）。问题是 T1 在同一事务内两次读同一数据结果不同——这是不可重复读。

B 正确：不可重复读。选 B。

A 错：丢失修改是写覆盖问题，这里是读不一致。

C 错：读脏数据是读了未提交的数据，这里 T2 已提交。

D 错：幻影读是因 INSERT/DELETE 导致查询结果集行数变化，这里是同一行值变了。

不可重复读 = 同一事务里读两次，值变了（别人提交了修改）。脏读 = 读到别人没提交的。幻影读 = 行数变了。

第 9 题

答案：B

解析：

一次封锁法：事务一次性把所有要用的数据全部加锁，要么全锁到要么不锁。

A 的描述本身正确（一次封锁法确实要求一次全部加锁），B 也正确（预防死锁但降低并发度）。本题存在 A、B 双正确的设计瑕疵，标准答案给 B，因为 B 同时点出了"预防死锁"的目的和"降低并发度"的代价，更全面。

C 错：一次封锁法不容易实现——因为要预先知道事务访问哪些数据，这很难。

D 错：一次封锁法必须知道事务要访问哪些数据才能一次全锁，恰恰这是它的难点。

一次封锁法 = "要啥先一次性全锁住再干"。能防死锁，但难点是要提前知道用啥数据，且并发度低。

第 10 题

答案：B

解析：

售票事务：查询余票 $\rightarrow$ 余票减 1 $\rightarrow$ 生成订单。用的是一级封锁协议。

一级封锁协议：修改前加 X 锁直到事务结束，读不加锁。

问题在"查询余票"这一步没加锁：

- T1 查询余票=1（没加锁）
- T2 查询余票=1（也没加锁）
- T1 减 1 写回余票=0（加 X 锁）
- T2 减 1 写回余票=-1（加 X 锁，但基于旧的 1） $\rightarrow$ 超卖

B 正确：一级封锁协议不能防止超卖，因为查询余票时没加锁，可能读到过时数据，导致两个事务都以为还有票。选 B。

A 错：一级封锁协议不够，读不加锁导致读到过时余票。

C 错：三级封锁协议能保证可重复读，但售票问题核心是"读-改-写"的原子性，关键是查询时就要加锁，三级确实能解决，但 B 更准确指出了一级的不足。



D 错：二级封锁协议读前加 S 锁读完释放，能防脏读但不能防读到过时数据后另一事务修改——不可重复读问题仍在，不一定解决超卖。

售票超卖的根本原因：查询时没加锁，多个事务同时读到同一个余票值。解决要在查询时就加锁（S 锁持续到事务结束 = 三级封锁协议）。

答案汇总

1.B 2.C 3.B 4.C 5.B 6.A 7.B 8.B 9.B 10.B

